Architecting Agentic ERP Interfaces: A Comprehensive Technical Framework for Implementing NetSuite AI Connector Service (ACS) Custom Tools Integration with ChatGPT

Executive Summary

The convergence of Enterprise Resource Planning (ERP) systems with Generative Artificial Intelligence (AI) marks a pivotal evolution in corporate information architecture. We are transitioning from a paradigm of *Direct Human Interaction*—where users manually navigate menus, execute reports, and update records—to *Agentic Interaction*, where Large Language Models (LLMs) like ChatGPT act as intelligent intermediaries. In this new architecture, the ERP does not merely store data; it exposes "tools"—discrete, executable functions—that an AI agent can invoke to perform work on behalf of a user.

In the NetSuite ecosystem, this capability is realized through the **NetSuite AI Connector Service (ACS)** and the implementation of **Custom Tool Scripts**. These scripts serve as the bridge between the probabilistic intent of an LLM and the deterministic logic of the SuiteScript 2.1 runtime engine.[1]

This report provides an exhaustive, expert-level technical guide for architects and developers tasked with building, deploying, and integrating these Custom Tools. It covers the entire lifecycle: from the initialization of a compliant **SuiteCloud Development Framework (SDF)** environment in Visual Studio Code, through the intricate security configuration of **OAuth 2.0 Machine-to-Machine (M2M)** authentication, to the final deployment and linkage with ChatGPT via the Model Context Protocol (MCP).

1. The Architectural Landscape: AI Connector Service and Custom Tools

Before interacting with code, it is essential to understand the architectural components that facilitate secure AI-ERP communication. The NetSuite AI Connector Service acts as a managed gateway, translating requests from external AI clients (like ChatGPT or Claude) into internal SuiteScript executions.

1.1 The Model Context Protocol (MCP)

The underlying communication standard is the **Model Context Protocol (MCP)**. Unlike traditional REST APIs, which are resource-oriented (e.g., GET /customer/5), MCP is tool-oriented. It allows the NetSuite host to broadcast a "manifest" of available capabilities (Tools) to the AI client. When a user asks ChatGPT to "Check inventory for Item X," the AI does not query a database directly; it selects the appropriate tool from the manifest, constructs a JSON payload matching the tool's schema, and sends it to the ACS endpoint.[3]

1.2 The Role of Custom Tool Scripts

The **Custom Tool** is a specific script type in SuiteScript 2.1 designed exclusively for this purpose. It differs from Suitelets or RESTlets in several critical ways:

- **No HTTP Context:** It does not handle HTTP request/response objects directly.
- **JSON Schema Enforcement:** Every tool must be paired with a rigorous JSON schema defining inputs and outputs.
- **Stateless Execution:** Each invocation is an isolated transaction.[4]

The successful implementation of an "ACS Custom Tool Script Project" requires a disciplined approach to project structure, security, and schema definition, which we will detail in the subsequent sections.

2. Development Environment and Infrastructure Prerequisites

Building for the NetSuite AI Connector requires a precise local development environment. The SuiteCloud Development Framework (SDF) relies on a Java/Node.js stack to manage the synchronization between the local file system and the NetSuite cloud.

2.1 Runtime Dependencies

The SuiteCloud CLI (Command Line Interface), which powers the Visual Studio Code extension, has strict version requirements. Mismatches here are the most common cause of deployment failures.

Component	Requirement	Context & Justification
Java Development Kit (JDK)	**Oracle JDK 17** or **JDK 21** (64-bit)	The core SDF deployment logic is Java-based. Recent updates have deprecated JDK 11 and JDK 8. Using older versions will result in validation errors during the

		project:deploy phase.[6]
Node.js	**Version 20 LTS** or **22 LTS**	The SuiteCloud CLI for Node.js (@oracle/suitecloud-cli) orchestrates the workflow. The CLI wrapper requires a modern Node runtime to handle asynchronous file operations and token management.[6]
SuiteCloud CLI	**Latest Version**	Installed globally via npm to ensure consistency across multiple projects.

To install the CLI globally, execute the following in your terminal:

```
Bash

npm install -g @oracle/suitecloud-cli
```

Verification: Run suitecloud --version to confirm the installation and path visibility.

2.2 Visual Studio Code Configuration

Visual Studio Code (VS Code) is the industry-standard IDE for SuiteScript development.

1. **Extension Installation:** Navigate to the VS Code Marketplace and install the **"SuiteCloud Extension for Visual Studio Code"** published by Oracle.[9]
2. **Workspace Settings:** Upon installation, configure the extension to use the globally installed CLI. This prevents "version drift" where the IDE uses an embedded, outdated CLI version while your terminal uses a newer one.

2.3 NetSuite Account Preparation

The target NetSuite account must have specific features enabled to support SDF and AI integrations. An Administrator must navigate to *Setup > Company > Enable Features* and activate:

- **SuiteCloud Development Framework:** Enables the deployment engine.[6]

- **Client SuiteScript** and **Server SuiteScript:** Required for the Custom Tool runtime.[10]
- **OAuth 2.0:** Mandatory for the AI Connector authentication.[7]
- **Token-Based Authentication:** Required for legacy support and internal handshake mechanisms.[10]
- **AI Connector Service:** (If available in your region) This feature flag specifically enables the MCP endpoints.

3. Security Architecture: OAuth 2.0 Machine-to-Machine (M2M) Setup

The user query specifically requests details on "tokens" and "appid". In the context of a modern, automated "Custom tool script project," relying on browser-based authentication (where a developer logs in via a popup) is insufficient for production pipelines and creates instability in AI connections. The robust standard is **OAuth 2.0 Machine-to-Machine (M2M)** authentication.[9]

This process involves a cryptographic handshake using an RSA key pair, an Integration Record (Application ID), and a Certificate Mapping.

3.1 Step 1: Generating the Cryptographic Keys

You must generate a 2048-bit RSA key pair. The **Private Key** resides on your local machine (or CI/CD secrets vault), and the **Public Certificate** is uploaded to NetSuite.

Execute the following commands in a secure terminal (Git Bash or Terminal):

Bash

```
# 1. Generate the Private Key
openssl genrsa -out netsuite-private.pem 2048

# 2. Extract the Public Certificate (Self-Signed)
openssl req -new -x509 -key netsuite-private.pem -out netsuite-certificate.pem -days 365
```

Note: When prompted for certificate details (Country, State, etc.), provided values are for identification only and do not affect the technical validity of the key.[11]

3.2 Step 2: Creating the Integration Record (The AppID)

The "AppID" (or Client ID) identifies the application attempting to connect.

1. Navigate to **Setup > Integration > Manage Integrations > New**.
2. **Name:** "SuiteCloud CLI Integration (M2M)".
3. **State:** Enabled.
4. **Authentication Tab:**
    - Check **OAuth 2.0**.
    - Check **Client Credentials (Machine-to-Machine) Grant**. *This is the critical setting that differentiates M2M from user-interactive flows*.[11]
5. **Save.**
6. **Copy Data:** NetSuite will display the **Client ID** (AppID) and **Client Secret**. Store these securely; the secret is shown only once.

3.3 Step 3: Certificate Mapping (The Handshake)

You must now tell NetSuite to trust any request signed by your Private Key.

1. Navigate to **Setup > Integration > OAuth 2.0 Client Credentials Setup**.
2. Click **Create New**.
3. **Entity:** Select the NetSuite User (e.g., a dedicated "AI Service Account" or Administrator) who will effectively "run" the deployments.
4. **Role:** Select "Administrator" or "SDF Developer".
5. **Application:** Select the Integration Record created in Step 3.2.
6. **Certificate:** Upload the netsuite-certificate.pem file generated in Step 3.1.
7. **Save.**
8. **Copy the Certificate ID:** NetSuite generates a unique string (e.g., custcert_12345...). You will need this for the VS Code setup.[11]

3.4 Step 4: Configuring Visual Studio Code

With the artifacts generated, you configure the IDE to authenticate.

1. Open the Command Palette (Ctrl+Shift+P or Cmd+Shift+P).
2. Type and select **SuiteCloud: Set Up Account**.
3. Choose **Machine-to-Machine** authentication.
4. **Account ID:** Enter your NetSuite Account ID.
    - *Tip:* Found in the URL (e.g., 1234567 or 1234567-sb1). Sandbox accounts must use hyphens, not underscores, in CLI contexts.[14]
5. **Certificate ID:** Enter the ID from Step 3.3.
6. **Private Key Path:** Enter the absolute path to netsuite-private.pem.
7. **Authentication ID (AuthID):** Create a memorable alias (e.g., MyProduction_M2M).

Validation: The extension will attempt to sign a JWT with your private key and exchange it for an access token. A success message indicates the pipeline is secure.

4. Initializing the SuiteCloud Project

NetSuite offers two primary project structures: **Account Customization Projects (ACP)** and **SuiteApp Projects**. The choice fundamentally alters how your AI tools are namespaced and accessed.

4.1 Comparison of Project Types

Feature	Account Customization Project (ACP)	SuiteApp Project (SDF SuiteApp)
Purpose	One-off customizations for a specific account.	Distributable applications or strictly isolated internal modules.
Namespacing	None (uses global customscript prefix).	Enforced Namespacing (customscript_publisherid_projectid).
AI Endpoint	Accessible via /all endpoint.	Accessible via /suiteapp/<appid> endpoint.
Portability	Low (hard to move between accounts cleanly).	High (designed for installation/upgrades).
Recommendation	Use for quick prototypes or single-instance fixes.	**Recommended for AI Tools** to maintain clean separation and security boundaries.[16]

4.2 Creating an Account Customization Project (ACP)

1. Open Command Palette: SuiteCloud: Create Project.
2. Select **Account Customization Project**.
3. **Project Name:** ACS_Custom_Tool_ACP.
4. **Include Unit Testing:** Select No (unless you have a Jest setup ready).
5. VS Code generates a folder structure mimicking the NetSuite File Cabinet (src/FileCabinet/SuiteScripts).[18]

4.3 Creating a SuiteApp Project (The "ACS" Standard)

This is the preferred method for an "ACS Custom tool script project" as it creates a dedicated

"Application ID" which is crucial for the AI Connector URL.

1. Open Command Palette: SuiteCloud: Create Project.
2. Select **SuiteApp Project**.
3. **Publisher ID:** Enter your SDN Publisher ID (e.g., com.acme) or a placeholder if internal (com.myorg).
4. **Project ID:** Enter a unique identifier (e.g., chatgpttools).
    - *Note:* The Publisher ID + Project ID forms the **Application ID** (com.myorg.chatgpttools).[19]
5. **Project Name:** ChatGPT Integration Tools.
6. **Version:** 1.0.0.
7. VS Code generates a strictly namespaced structure.

5. Detailed Project Anatomy and Configuration

Once the project is initialized, you must configure the manifest to declare the necessary feature dependencies. The AI Connector relies on Server-Side Scripting; failing to declare this will cause deployment validation to fail.

5.1 The manifest.xml

Located at the root of src, this file controls the project metadata.

XML

```
<manifest projecttype="SUITEAPP">
  <publisherid>com.myorg</publisherid>
  <projectid>chatgpttools</projectid>
  <projectname>ChatGPT Integration Tools</projectname>
  <projectversion>1.0.0</projectversion>
  <dependencies>
    <features>
      <feature required="true">SERVERSIDESCRIPTING</feature>
      <feature required="true">CUSTOMRECORDS</feature>
    </features>
  </dependencies>
</manifest>
```

Insight: The SERVERSIDESCRIPTING feature dependency is non-negotiable for Custom Tools.[20]

5.2 The Directory Structure

A properly architected AI project should follow this hierarchy:

- src/
  - manifest.xml
  - deploy.xml
  - FileCabinet/
    - SuiteApps/
      - com.myorg.chatgpttools/
        - SuiteScripts/ (Contains the.js logic)
        - Schemas/ (Contains the.json definitions)
  - Objects/
    - customtool_my_tool.xml (The SDF Object definition)

6. Implementing the Custom Tool Script (SuiteScript 2.1)

The **Custom Tool** script type is unique. It exports a set of functions that map directly to the tools exposed to the AI.

6.1 Script Requirements

- **Version:** Must use @NApiVersion 2.1 to support modern JavaScript features.[21]
- **Module Scope:** Must be Public if part of a SuiteApp.
- **Restricted Modules:** You cannot use N/http or N/https to call external APIs. This prevents the AI from using NetSuite as a proxy to attack other systems. All logic must operate on *internal* NetSuite data (using N/search, N/record, N/query).[4]

6.2 Code Example: Inventory Checker and Case Creator

Create a file chatgpt_integration.js in your SuiteScripts folder.

JavaScript

```
/**
 * @NApiVersion 2.1
 * @NScriptType CustomTool
 * @NModuleScope Public
 */
define(['N/search', 'N/record', 'N/format'], (search, record, format) => {
```

```
    /**
     * Tool 1: Check Inventory
     * Checks stock levels for a given item name.
     */
    const checkInventory = (input) => {
        try {
            // Input validation
            if (!input.itemName) {
                return { error: "Item Name is required." };
            }

            // Execute SuiteScript Search
            const itemSearch = search.create({
                type: search.Type.ITEM,
                filters: ['itemid', 'is', input.itemName],
                    'AND',
                    ['isinactive', 'is', 'F'],
                columns: ['displayname', 'quantityavailable', 'baseprice']
            });

            const results = itemSearch.run().getRange({ start: 0, end: 1 });

            if (results.length === 0) {
                return {
                    result: JSON.stringify({
                        found: false,
                        message: `Item '${input.itemName}' not found.`
                    })
                };
            }

            const item = results;
            const data = {
                found: true,
                name: item.getValue('displayname'),
                quantity: item.getValue('quantityavailable'),
                price: item.getValue('baseprice')
            };

            // Return stringified JSON as 'result'
            return {
                result: JSON.stringify(data)
```

```
            };

        } catch (e) {
            log.error({ title: 'Inventory Check Failed', details: e });
            return { error: `System Error: ${e.message}` };
        }
    };

    /**
     * Tool 2: Create Support Ticket
     * Creates a Case record from user input.
     */
    const createTicket = (input) => {
        try {
            const ticket = record.create({ type: record.Type.SUPPORT_CASE });
            ticket.setValue({ fieldId: 'title', value: input.subject });
            ticket.setValue({ fieldId: 'incomingmessage', value: input.description });
            const id = ticket.save();

            return {
                result: JSON.stringify({
                    success: true,
                    ticketId: id,
                    message: "Support case created successfully."
                })
            };
        } catch (e) {
            return { error: `Failed to create ticket: ${e.message}` };
        }
    };

    // Export functions mapped to tool names
    return {
        checkInventory: checkInventory,
        createTicket: createTicket
    };
});
```

Insight: The return object { result: string, error: string } is the strict contract required by the AI Connector. The result field *must* be a string (often stringified JSON) because the MCP layer treats the output as text content to be fed back into the LLM's context window.[20]

7. The Cognitive Interface: JSON Schema Definition

For ChatGPT to "understand" how to use the code written above, you must provide a definition file. This JSON schema is not just validation; it is **prompt engineering**. The descriptions provided in this file are read by the AI to determine *when* to call the tool and *what* data to extract from the user's conversation.

7.1 The Schema File Structure

Create chatgpt_integration_schema.json in the same folder.

Critical Requirement for ChatGPT: The nullable array. Recent implementations of the ChatGPT integration via MCP require that fields—even those that are required—have their nullability explicitly defined. Failing to include the nullable array (even if empty) often causes the GPT action to fail validation.[5]

JSON

```
{
  "tools":,
            "nullable":
        },
        "outputSchema": {
            "type": "object",
            "properties": {
                "result": {
                    "type": "string",
                    "description": "JSON string containing inventory details."
                },
                "error": {
                    "type": "string",
                    "description": "Error message if the operation fails."
                }
            },
            "nullable": ["error"]
        }
    },
    {
        "name": "createTicket",
        "description": "Creates a new support case in the system. Use this when the user reports a
```

```
problem or requests help.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "subject": {
              "type": "string",
              "description": "A brief summary of the issue."
            },
            "description": {
              "type": "string",
              "description": "The full details of the user's problem."
            }
          },
          "required": ["subject", "description"],
          "nullable":
        },
        "outputSchema": {
          "type": "object",
          "properties": {
            "result": { "type": "string" },
            "error": { "type": "string" }
          },
          "nullable": ["error"]
        }
      }
  ]
}
```

8. SDF Custom Object Definition

To deploy these files as a recognized "Custom Tool" object in NetSuite, you must define the metadata in an XML file within the src/Objects directory.

8.1 The customtool Object

Create customtool_chatgpt_integration.xml. This file links the Script and the Schema and defines the permissions.

The <exposeto3rdpartyagents> Tag:

This is the switch that enables the tool for the AI Connector. **Crucial Warning:** Ensure the tag is plural (agents), as some older documentation might reference the singular.

XML

```
<tool scriptid="customtool_chatgpt_integration">
  <name>ChatGPT Integration Tools</name>
  <scriptfile></scriptfile>
  <rpcschema></rpcschema>

  <exposeto3rdpartyagents>T</exposeto3rdpartyagents>

  <permissions>
    <permission>
      <permkey>LIST_ITEM</permkey>
      <permlevel>VIEW</permlevel>
    </permission>
    <permission>
      <permkey>LIST_CASE</permkey>
      <permlevel>FULL</permlevel>
    </permission>
  </permissions>
</tool>
```

Security Insight: The permissions defined here are the *maximum* capabilities of the tool. However, at runtime, these are intersected with the permissions of the User/Role authenticated via OAuth. Both must permit the action for it to succeed.[23]

9. Deployment Strategy to Target Account

With the file cabinet resources and object definitions created, the final step in the local environment is deployment.

9.1 The deploy.xml Manifest

Update your src/deploy.xml to include the new files.

XML

```
<deploy>
```

```
    <files>
        <path>~/SuiteApps/com.myorg.chatgpttools/SuiteScripts/chatgpt_integration.js</path>

<path>~/SuiteApps/com.myorg.chatgpttools/SuiteScripts/chatgpt_integration_schema.json</path>
    </files>
    <objects>
        <path>~/Objects/customtool_chatgpt_integration.xml</path>
    </objects>
</deploy>
```

9.2 Validation and Deployment

Execute the deployment using the VS Code Command Palette.

1. **Validation:** SuiteCloud: Validate Project.
    - This parses the XML and checks for circular dependencies or missing features.
    - *Common Error:* "Account Specific Value" warnings. If your XML references specific IDs (like a specific Customer ID), SDF will flag this. Use script IDs instead where possible.[25]
2. **Deployment:** SuiteCloud: Deploy Project.
    - This zips the project and uploads it to the account authenticated via the M2M AuthID.
    - *Output:* Watch the console for "Deployment Finished Successfully".

10. Integration with ChatGPT (The Final Link)

Once deployed, the Custom Tool exists in NetSuite, but ChatGPT does not yet know how to reach it. We must configure the connection.

10.1 Constructing the AI Connector URL

The endpoint URL depends on whether you built an Account Customization Project (ACP) or a SuiteApp.

Project Type	URL Structure	Example
SuiteApp	/suiteapp/<publisher>.<project>	https://1234567.suitetalk.api.netsuite.com/services/mcp/v1/suiteapp/com.myorg.chatgpttools
ACP	/all	https://1234567.suitetalk.api.netsuite.com/services/mcp

		/v1/all

Note: Using the /all endpoint exposes *every* custom tool in the account. The SuiteApp-specific URL is cleaner as it only exposes the tools defined in your specific project.[27]

10.2 Configuring the Connector in ChatGPT

This step depends on whether you are using **ChatGPT Enterprise/Team** (which may have a native "NetSuite" connector integration) or building a **Custom GPT**.

For Custom GPTs (GPT Builder):

1. **Create a GPT:** Go to "Explore GPTs" > "Create".
2. **Instructions:** "You are a NetSuite assistant. Use the provided actions to query inventory and manage cases."
3. **Configure Action:**
   - **Authentication:** Select **OAuth 2.0**.
   - **Client ID:** The AppID from Step 3.2.
   - **Client Secret:** The Secret from Step 3.2.
   - **Authorization URL:** https://<ACCOUNT_ID>.app.netsuite.com/app/login/oauth2/authorize.nl
   - **Token URL:** https://<ACCOUNT_ID>.suitetalk.api.netsuite.com/services/rest/auth/oauth2/v1/token
   - **Scope:** rest_webservices (Standard scope for SuiteTalk/MCP).
4. **Schema Import:**
   - *Challenge:* GPT Actions typically expect an OpenAPI (Swagger) spec, whereas NetSuite ACS uses MCP.
   - *Solution:* If using the **NetSuite AI Connector** (official feature), you simply provide the **Server URL** (from 10.1) into the "Connectors" configuration screen of your AI client, provided it supports the Model Context Protocol. If building a raw GPT Action, you would need to manually convert your json schema into an OpenAPI definition that points to the NetSuite endpoint.

Note on Native AI Connector: If your ChatGPT instance supports the "NetSuite" connector natively (often found in the "Connectors" or "Integrations" menu of enterprise AI tools), you simply:

1. Select "NetSuite".
2. Enter the **Server URL**.
3. Log in via the OAuth prompt.
4. The tools defined in your schema automatically populate the agent's context.

11. Conclusion

Implementing an ACS Custom Tool script project transforms NetSuite from a passive system of record into an active participant in business dialogue. By rigorously following the **SDF project structure**, enforcing **OAuth 2.0 M2M security**, and defining clear **JSON Schemas**, developers can safely expose ERP logic to GenAI models.

This architecture—separating the *definition* of work (Schema) from the *execution* of work (SuiteScript)—is the foundation of the next decade of ERP customization. It allows organizations to leverage the reasoning capabilities of ChatGPT while maintaining the transactional integrity and security governance of NetSuite.

Works cited

1. Artificial Intelligence in NetSuite, accessed February 9, 2026, https://www.netsuite.com/portal/products/artificial-intelligence-ai.shtml
2. AI in ERP: Next-Gen ERP Solutions - NetSuite, accessed February 9, 2026, https://www.netsuite.com/portal/resource/articles/erp/ai-erp.shtml
3. NetSuite MCP Challenge: Implementation Case Study & Results - Folio3, accessed February 9, 2026, https://netsuite.folio3.com/blog/netsuite-mcp-challenge-implementation-case-study-results/
4. NetSuite Applications Suite - SuiteScript 2.1 Custom Tool Script Type - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_1185045525.html
5. NetSuite Applications Suite - SuiteScript 2.1 Custom Tool Script Type Reference, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_0724071739.html
6. SuiteCloud CLI for Node.js Installation Prerequisites - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1558708810.html
7. NetSuite Applications Suite - SuiteCloud CLI for Java Installation Prerequisites, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1489072297.html
8. @oracle/suitecloud-cli - npm, accessed February 9, 2026, https://www.npmjs.com/package/@oracle/suitecloud-cli
9. SuiteCloud Extension for Visual Studio Code - Visual Studio ..., accessed February 9, 2026, https://marketplace.visualstudio.com/items?itemName=Oracle.suitecloud-vscode-extension
10. How to get your NetSuite API Key - Apideck, accessed February 9, 2026, https://www.apideck.com/blog/how-to-get-your-netsuite-api-key
11. NetSuite Applications Suite - OAuth 2.0 Authentication for SuiteCloud SDK,

accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_0422011927.html
12. M2M OAuth2.0 account configuration in NetSuite - SnapLogic Documentation, accessed February 9, 2026, https://docs.snaplogic.com/snaps/snaps-enterprise/sp-netsuite-rest/acct-config-nsr-m2m-oauth2.html
13. NetSuite Applications Suite - account:setup:ci - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_81134826821.html
14. accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1498754928.html#:~:text=You%20can%20find%20your%20NetSuite,an%20underscore%2C%20for%20example%20123456_RP.
15. Finding Your NetSuite Account ID - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1498754928.html
16. NetSuite Applications Suite - SuiteCloud Project Types - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4706651345.html
17. NetSuite Applications Suite - Account Customization Projects - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/subsect_1510680449.html
18. NetSuite Applications Suite - Creating a SuiteCloud Project in ..., accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1629385060 15.html
19. NetSuite Applications Suite - SuiteApp Projects - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/subsect_1509931104.html
20. Building Custom Tools for NetSuite AI Connector: Development Guide - folio3, accessed February 9, 2026, https://netsuite.folio3.com/blog/building-custom-tools-for-netsuite-ai-connector-development-guide/
21. NetSuite Applications Suite - SuiteScript 2.1 Language Examples - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_156042699370.html
22. Has anyone created a custom tool that connects from chatGPT NetSuite Professionals #ai-netsuite, accessed February 9, 2026,

https://archive.netsuiteprofessionals.com/t/30250305/has-anyone-created-a-custom-tool-that-connects-from-chatgpt-
23. NetSuite Applications Suite - SuiteScript 2.1 Custom Tool Script Type Code Samples, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_0724092648.html
24. NetSuite Applications Suite - tool - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_0825085327.html?source=%3Aex%3Apw%3A%3A%3A%3A%3ATNS_SQL_2_D
25. NetSuite Applications Suite - deploy - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4788673412.html
26. Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1539789946.html
27. NetSuite Applications Suite - Connect to the NetSuite AI Connector ..., accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_0714082142.html